Guía de Referencia — JavaScript

Resumen de todo lo visto en el Módulo 2 (Clases 14 a 17): strings, arrays, objetos y funciones. Úsala como apoyo rápido mientras resuelves ejercicios.

---

1. Métodos de Strings

Estos métodos NO modifican el string original (los strings son inmutables en JavaScript): siempre devuelven un string NUEVO. Por eso hay que guardarlo en una variable o imprimirlo directamente.

toUpperCase()

Convierte todo el texto a mayúsculas.

```
const texto = "hola mundo"
console.log(texto.toUpperCase())
// "HOLA MUNDO"
```

toLowerCase()

Convierte todo el texto a minúsculas.

```
const texto = "HOLA MUNDO"
console.log(texto.toLowerCase())
// "hola mundo"
```

trim()

Elimina los espacios en blanco del inicio Y el final del texto.

```
const nombre = "   carlos  "
console.log(nombre.trim())
// "carlos"
```

trimStart() / trimEnd()

Igual que trim(), pero solo limpia espacios al inicio (trimStart) o solo al final (trimEnd).

```
console.log("  hola  ".trimStart())  // "hola  "
console.log("  hola  ".trimEnd())    // "  hola"
```

includes()

Devuelve true o false según si el texto contiene la palabra/letra buscada.

```
const fruta = "banana"
console.log(fruta.includes("b"))
// true
```

startsWith() / endsWith()

Verifican si el texto empieza o termina con cierta palabra/letra.

```
const fruta = "naranja"
console.log(fruta.startsWith("na"))  // true
console.log(fruta.endsWith("ja"))    // true
```

indexOf() / lastIndexOf()

Devuelven la POSICIÓN (índice) de la primera o la última coincidencia. Si no la encuentra, devuelven -1.

```
const fruta = "naranja"
console.log(fruta.indexOf("n"))      // 0
console.log(fruta.lastIndexOf("a"))  // 6
```

search()

Busca una coincidencia usando una expresión regular (regex) y devuelve su posición.

```
const fruta = "naranja"
console.log(fruta.search(/n/))
// 0
```

replace() / replaceAll()

replace cambia solo la PRIMERA coincidencia; replaceAll cambia TODAS las coincidencias.

```
const mensaje = "yo no te amo"
console.log(mensaje.replace("o", "a"))     // "ya no te amo"
console.log(mensaje.replaceAll("o", "a"))  // "ya na te ama"
```

split()

Convierte un string en un ARRAY, cortando el texto donde indiques el separador.

```
const alumnos = "Manuel-Pepe-Maria"
console.log(alumnos.split("-"))
// ["Manuel", "Pepe", "Maria"]
```

concat()

Une (concatena) dos o más strings en uno solo.

```
const t1 = "Ella es"
const t2 = "la mejor"
console.log(t1.concat(" ", t2))
// "Ella es la mejor"
```

slice()

Extrae una parte (porción) del string. Si solo pones un número, corta desde ahí hasta el final.

```
const e = "programacion"
console.log(e.slice(3))
// "gramacion"
```

length

Propiedad (no método, sin paréntesis) que devuelve la cantidad de caracteres del string.

```
const t = "hola mundo"
console.log(t.length)
// 10
```

charAt() / [índice] / at()

Tres formas de acceder a un carácter según su posición. at() permite índices negativos para contar desde el final.

```
const t = "hola mundo"
console.log(t.charAt(2))  // "l"
console.log(t[0])         // "h"
console.log(t.at(-2))     // "d"
```

localeCompare()

Compara dos strings alfabéticamente. Devuelve -1 si va antes, 1 si va después, 0 si son iguales.

```
console.log("z".localeCompare("a"))
// 1 (porque "z" va después de "a")
```

2. Métodos de Arrays

Algunos métodos de arrays SÍ modifican (mutan) el array original — se marcan en **rojo como "MUTADOR"**. Los demás devuelven un array/valor nuevo sin tocar el original.

push() / unshift()

MUTADOR — modifica el array original

Agregan elementos: push() al FINAL, unshift() al INICIO. Se pueden agregar varios a la vez.

```
const frutas = ["manzana", "pera"]
frutas.push("kiwi")
frutas.unshift("naranja")
console.log(frutas)
// ["naranja", "manzana", "pera", "kiwi"]
```

pop() / shift()

MUTADOR — modifica el array original

Eliminan elementos: pop() elimina el ÚLTIMO, shift() elimina el PRIMERO.

```
const frutas = ["naranja", "manzana", "pera"]
frutas.pop()    // elimina "pera"
frutas.shift()  // elimina "naranja"
console.log(frutas)  // ["manzana"]
```

reverse()

MUTADOR — modifica el array original

Invierte el orden de los elementos del array.

```
const numeros = [1, 2, 3]
console.log(numeros.reverse())
// [3, 2, 1]
```

slice(inicio, fin)

Extrae una porción del array SIN modificar el original. El índice 'fin' no se incluye.

```
const numeros = [1, 2, 3, 4]
const porcion = numeros.slice(1, 3)
console.log(porcion)  // [2, 3]
console.log(numeros)  // [1, 2, 3, 4]  (no cambió)
```

concat()

Une dos o más arrays en uno nuevo.

```
const a1 = [1, 2, 3]
const a2 = [4, 5, 6]
console.log(a1.concat(a2))
// [1, 2, 3, 4, 5, 6]
```

join(separador)

Convierte el array en un STRING, uniendo sus elementos con el separador indicado.

```
console.log(["Lima", "Piura", "Cuzco"].join(", "))
// "Lima, Piura, Cuzco"
```

includes()

Devuelve true o false según si el array contiene el valor buscado.

```
const roles = ["user", "guest"]
console.log(roles.includes("admin"))
// false
```

at(índice)

Accede a un elemento por su posición. Permite índices negativos para contar desde el final.

```
const numeros = [5, 6, 7, 8, 9]
console.log(numeros[2])     // 7
console.log(numeros.at(-1)) // 9
```

filter(callback)

Devuelve un NUEVO array solo con los elementos que cumplen la condición. No modifica el original.

```
const notas = [12, 8, 15, 10, 18]
const aprobados = notas.filter(n => n >= 11)
console.log(aprobados)  // [12, 15, 18]
```

map(callback)

Devuelve un NUEVO array transformando CADA elemento según la función. (A diferencia de filter, no selecciona: transforma todos).

```
const precios = [100, 200, 300]
console.log(precios.map(p => p * 0.9))
// [90, 180, 270]
```

forEach(callback)

Ejecuta una función por cada elemento del array. No devuelve nada nuevo (a diferencia de map/filter), se usa para EJECUTAR algo, no para transformar.

```
const notas = [10, 15, 8]
const resultado = { aprobados: 0, desaprobados: 0 }
notas.forEach(n => {
  if (n >= 11) { resultado.aprobados++ }
  else { resultado.desaprobados++ }
})
console.log(resultado)  // { aprobados: 1, desaprobados: 2 }
```

every() / some()

every: ¿TODOS los elementos cumplen la condición? some: ¿AL MENOS UNO la cumple?

```
const numeros = [1, 2, 43, 0, 3, 0]
console.log(numeros.every(n => n == 0))  // false
console.log(numeros.some(n => n == 0))   // true
```

find(callback)

Devuelve el PRIMER elemento que cumple la condición (a diferencia de filter, que devuelve TODOS). Si no encuentra nada, devuelve undefined.

```
const notas = [8, 15, 10, 18]
console.log(notas.find(n => n >= 11))
// 15 (el primero que cumple)
```

3. Objetos

Crear y acceder a un objeto

Un objeto agrupa datos relacionados con pares clave-valor (llaves {}). Se accede con punto o con corchetes.

```
const producto = {
  nombre: "Teclado",
  precio: 120,
  color: "Blanco"
}
console.log(producto.nombre)   // "Teclado"
console.log(producto["color"]) // "Blanco"
```

Objeto con cualquier tipo de dato

Una propiedad de un objeto puede ser un string, número, o incluso un array.

```
const fruta = {
  nombre: "Pera",
  pesoEnGr: 300,
  familia: ["fruta1", "fruta2"]
}
```

Array de objetos

Es la estructura más usada en la práctica: una lista donde cada elemento es un objeto con las mismas propiedades.

```
const alumnos = [
  { nombre: "Pepe", edad: 16, promedio: 13 },
  { nombre: "Maria", edad: 20, promedio: 11 },
]
// Acceder al primer alumno:
console.log(alumnos[0].nombre)  // "Pepe"
```

Combinando objetos con filter / map / forEach

Dentro del callback, accedes a las propiedades de cada objeto con punto.

```
const alumnos = [
  { nombre: "Pepe", edad: 16, promedio: 13 },
  { nombre: "Donatto", edad: 17, promedio: 17 },
]
const nombres = alumnos
  .filter(a => a.promedio >= 11)
  .map(a => a.nombre)
console.log(nombres)  // ["Pepe", "Donatto"]
```

4. Funciones

Función normal (declaración)

La forma clásica de definir una función reutilizable.

```
function areaTriangulo(base, altura) {
  console.log(base * altura / 2)
}
areaTriangulo(5, 2)  // 5
```

Arrow function

Forma más corta de escribir funciones, muy usada dentro de map/filter/forEach, y también sola.

```
const enviarMensaje = (mensaje) => {
  console.log(mensaje)
}
enviarMensaje("Hola")
```

Parámetros por defecto (opcionales)

Si no se envía un valor para ese parámetro, se usa el valor por defecto indicado.

```
function saludar(nombre = "Invitado") {
  console.log("Hola, " + nombre)
}
saludar()          // "Hola, Invitado"
saludar("Antonella") // "Hola, Antonella"
```

Operador ternario

Forma corta de escribir un if/else en una sola línea: condición ? si es verdad : si es falso.

```
const edad = 13
edad >= 18
  ? console.log("Eres mayor")
  : console.log("Eres menor")
```

return vs console.log

console.log solo IMPRIME el valor en consola. return ENTREGA el valor para que se pueda guardar en una variable o seguir usándolo fuera de la función.

```
function sumar(a, b) {
  return a + b
}
const resultado = sumar(2, 3)
console.log(resultado)  // 5
```

Funciones que combinan filter + map

Una función puede recibir un array y devolver (con return) un nuevo array ya filtrado y transformado.

```
function productosHasta(lista, dineroMaximo) {
  return lista
    .filter(p => p.precio <= dineroMaximo)
    .map(p => p.nombre)
}
```